

树上路径问题



树上路径问题，就是不停询问树上点 u 到点 v 之间的查询极值和最短路径等，与区间查询类似。

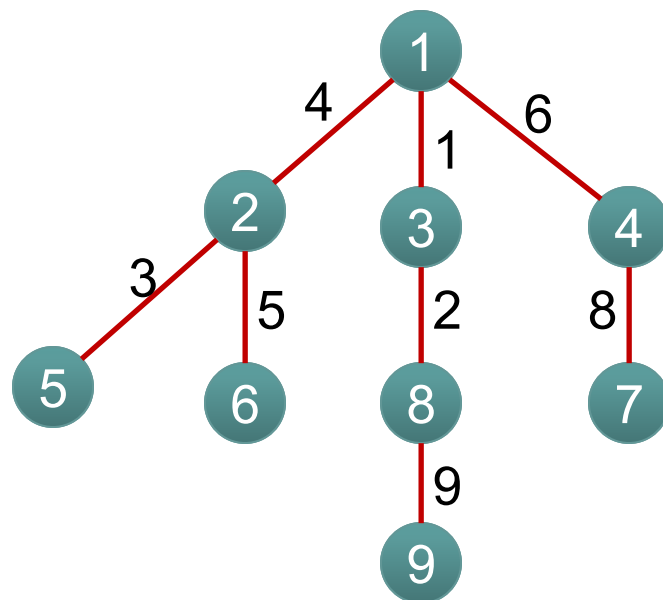
如下图：不停询问点对之间的距离

2---7

3---6

3---4

5---6



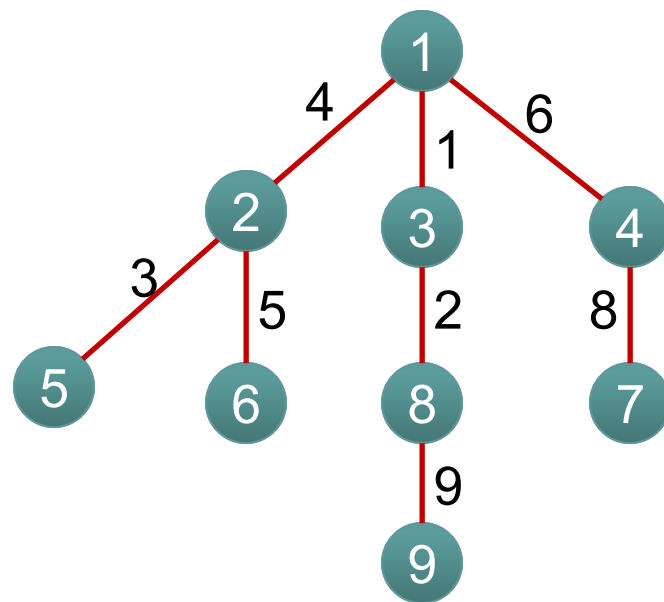
任意两点的最近公共祖先

最近公共祖先



什么是最近公共祖先？

在一棵没有环的树上，每个节点肯定有其父亲节点和祖先节点，而最近公共祖先，就是两个节点在这棵树上**深度最大的公共的祖先节点**。换句话说，就是两个点在这棵树上**距离最近的公共相同祖先节点**。所以**LCA**主要是用来处理当两个点仅有唯一一条确定的最短路径时的路径。



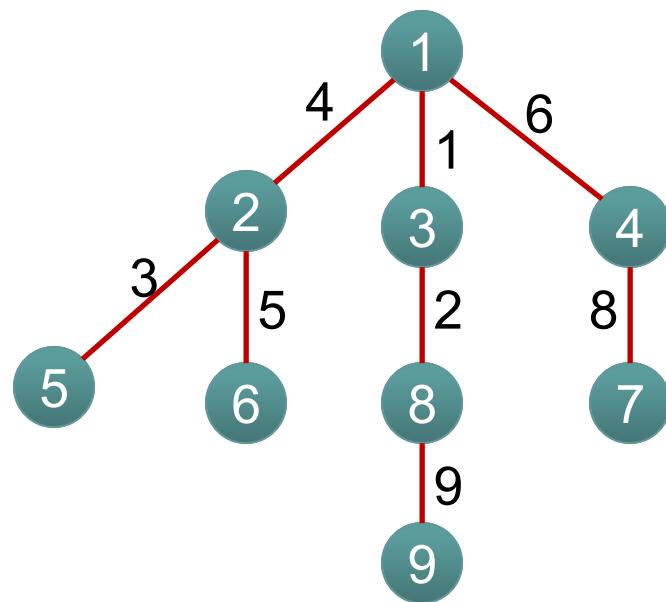
暴力求LCA



求a和b的公共祖先

- 1、先从a向上遍历，标记走过的每个节点，直到根
- 2、再从b向上遍历，访问到第一个被标记过的点，就是答案。

此方法适合询问次数极少的情况。若询问m比较大，容易超时。

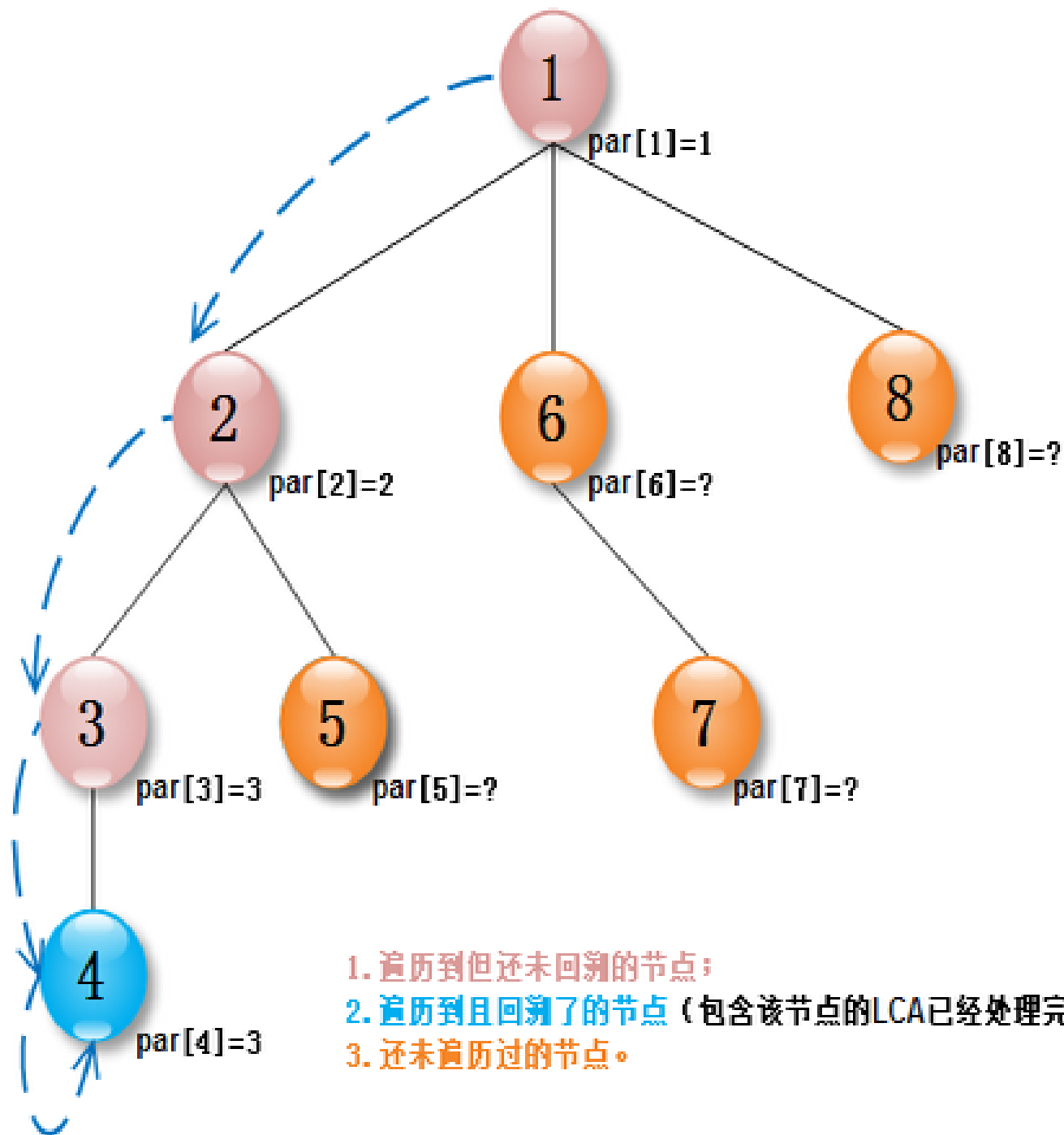
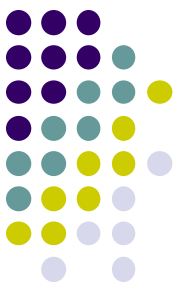


Tarjan求LCA

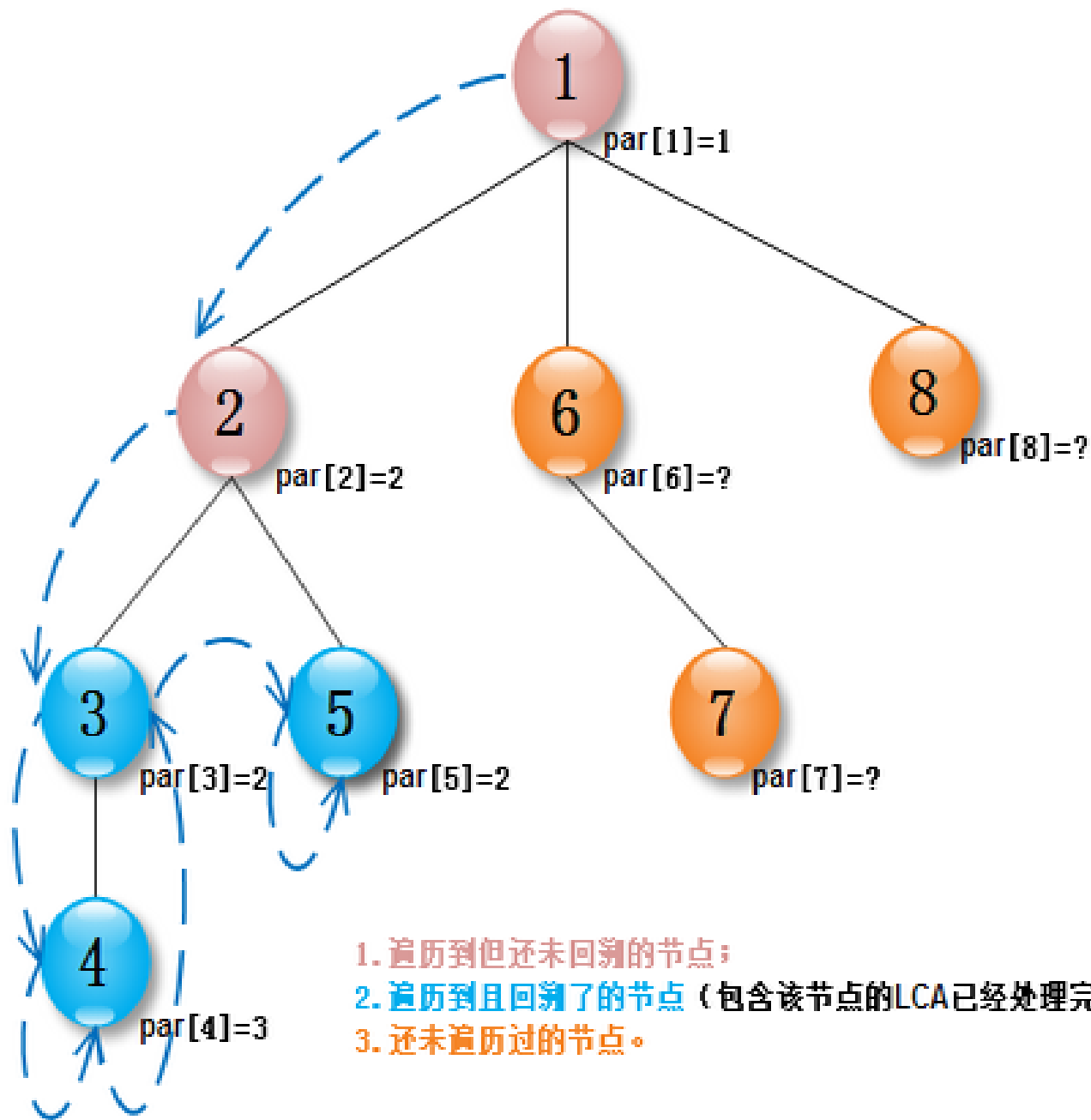
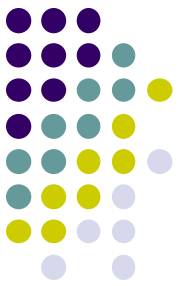


Tarjan求解LCA算法是一个离线的算法。即在一次遍历中把所有的询问一次性解决，时间复杂度是 $O(n+m)$ 。

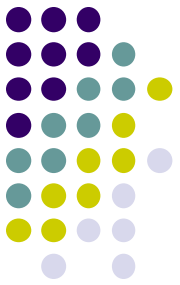
- 解题思路：利用dfs+并查集解决，利用dfs递归搜索所有点，搜索过的点做标记book[x],回溯时标记父亲fa[x]。对于每一个点，遍历其所有儿子后，处理与该点有关系的点y.如果y未访问，不管。如果y已经访问过，则y要么在链条顶上，要么在访问过的分支。



1. 遍历到但还未回溯的节点；
2. 遍历到且回溯了的节点（包含该节点的LCA已经处理完毕）；
3. 还未遍历过的节点。



1. 遍历到但还未回溯的节点；
2. 遍历到且回溯了的节点（包含该节点的LCA已经处理完毕）；
3. 还未遍历过的节点。



解题步骤：

1、从树的根节点(**root**)开始深搜

2对于新搜索到的一个节点 (**x**)，首先创建由这个结点构成的集合

(由**father[]**数组维护，**father[x]=x**，当前这个集合中只有元素**x**)

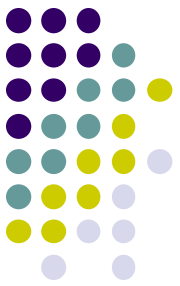
3、然后依次搜索并处理该节点包含的所有子树（搜索和处理是个递归的过程。

结合第5点理解：每搜索完一棵子树，则可确定子树内的**LCA**询问都已解，

其他的**LCA**询问的结果必然在这个子树之外。)

学习参考博客：

<http://www.cnblogs.com/JVxie/p/4854719.html>



```
void tarjan_lca(int x)
{
    father[x]=x;//构建以x为根的集合
    visit[x]=true;
    for(int i=head[x];i>0;i=e[i].next)//枚举相邻边，递归子树
    {
        if(!visit[e[i].to])//如果当前节点没有访问过
        {
            tarjan_lca(e[i].to);//递归子树
            father[e[i].to]=x;//回溯集合合并子树
        }
    }
    for(int i=head_q[x];i>0;i=e_q[i].next)//处理包含x点对询问且以遍历最近公共祖先
    {
        if(visit[e_q[i].to]) //如何当前询问对的节点遍历过
        {
            int lca=find(e_q[i].to);
            printf("%d-->%d: %d\n",x,e_q[i].to,lca);
        }
    }
}
```


RQM转LCA的弊端



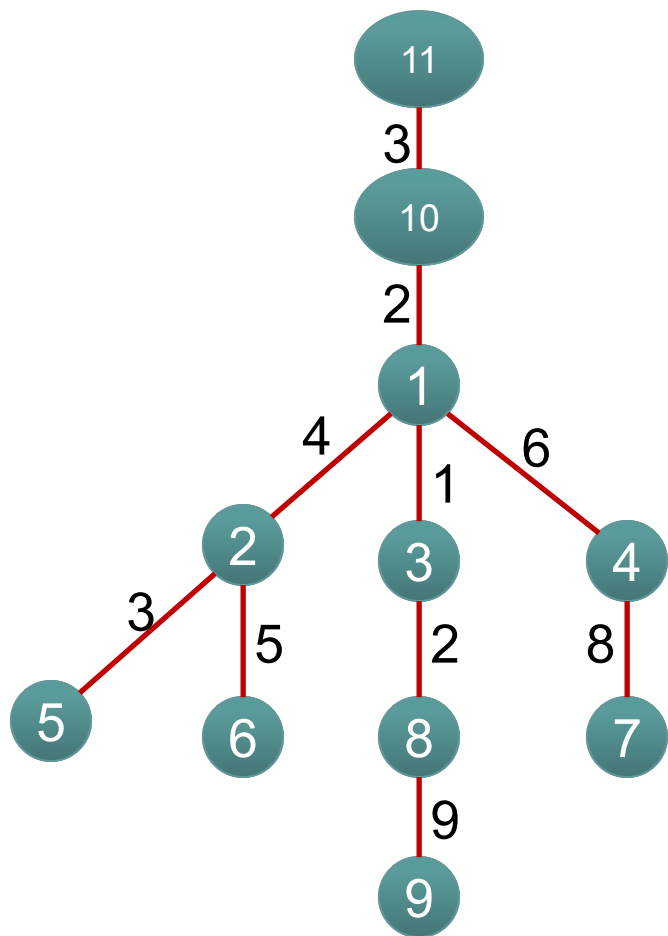
优势：算法思想简单，很容易快速求出 $u-v$ 的最近公共祖先节点

缺点：不能够求解 $u-v$ 路径上权值最小值或最大值问题。如2013 noip day1-3。

如何才能快速求出LCA且路径权值的最值问题呢？

树上倍增

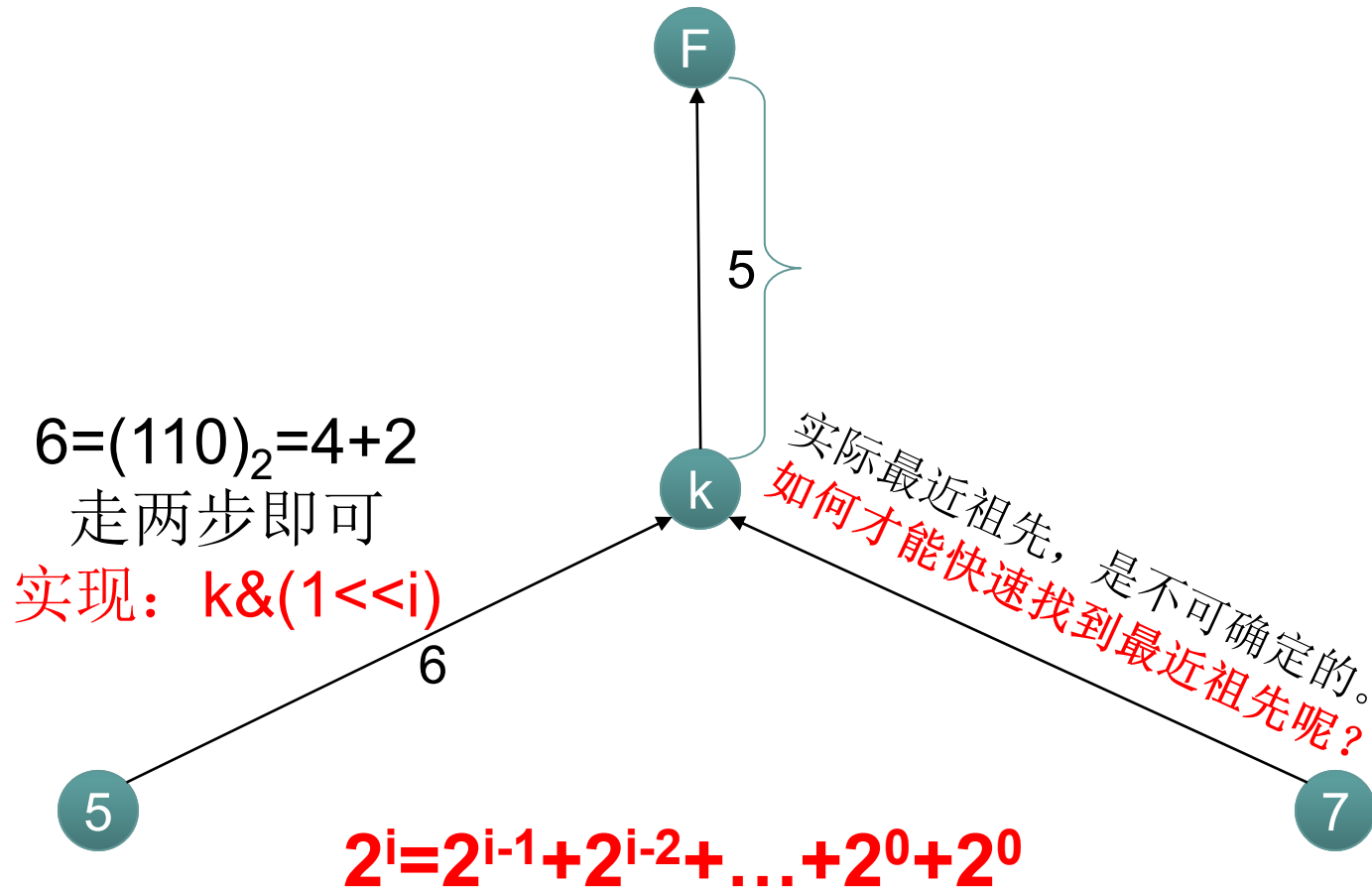
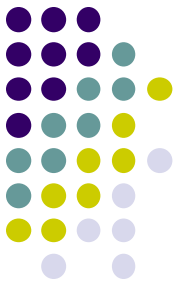
树上倍增



如何快速找到*i*点和*j*点的公共祖先

如何快速找到5和7的最近公共祖先？
朴素：从5和7开始依次向上遍历，最先找到的相同节点即为最近公共祖先。
怎么样优化朴素方法呢？

利用倍增思想，快速遍历



注: 当所求两点不在同一高度时?
先调整亮点到统一高度, 再向上倍增

倍增算法流程



- 1、dfs预处理出i点向上 2^j 的节点
- 2、倍增遍历，快速求出i和j点的最近公共祖先。
- 3、根据需求，倍增查询i和j到祖先间的值。

树上倍增



树上倍增算法思想：就是通过RMQ的思想，能够快速遍历到i点的祖先。于是采用RMQ的思想对u点的祖先预处理。

用 $fa[i][j]$ 表示i点的第 2^j 个祖先，快速实现上层祖先的查找，转移方程： $fa[i][j] = fa[fa[i][j-1]][j-1]$
 $dp_min[i][j]$ 表示从i点到第 2^j 个祖先的最小值，转移方程： $dp_min[i][j] = \min(dp[i][j-1], dp[fa[i][j-1]][j-1])$

Dfs预处理



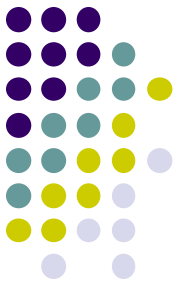
```
void dfs(int x)
{
    visit[x]=true;
    for(int i=1;(1<i)<=deep[x];i++)//这里在完成x点到它上层祖先点的预处理
    {
        fa[x][i]=fa[fa[x][i-1]][i-1];
        dp_min[x][i]=min(dp_min[x][i-1],dp_min[fa[x][i-1]][i-1]);
    }
    for(int i=head[x];i>0;i=edge[i].next)
    {
        if(!visit[edge[i].to])
        {
            fa[edge[i].to][0]=x;
            dp_min[edge[i].to][0]=edge[i].w;
            deep[edge[i].to]=deep[x]+1;
            dfs(edge[i].to);
        }
    }
}
```

求LCA

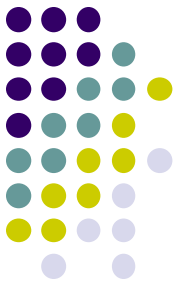


```
int lca(int x,int y)
{
    if(deep[x]<deep[y]) swap(x,y);
    int t=deep[x]-deep[y];//计算深度差
    for(int i=0;(1<<i)<=t;i++) if(t&(1<<i)) x=fa[x][i];//深度差调整到同一高度
    if(x==y) return x;//x和y在一条链上
    for(int i=17;i>=0;i--)//向上枚举找最近公共祖先，从根开始向下，思考为什么？
    {
        if(fa[x][i]!=fa[y][i])
        {
            x=fa[x][i];y=fa[y][i];
        }
    }
    return fa[x][0];
}
```

查询



```
int ask(int x,int f)
{
    int mm=0x7f7f7f;
    int t=deep[x]-deep[f];
    for(int i=0;(1<<i)<=t;i++)
    {
        if(t&(1<<i))
        {
            mm=min(mm,dp_min[x][i]);
            x=fa[x][i];
        }
    }
    return mm;
}
```

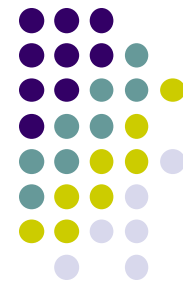



树上倍增需要注意的几个问题

- 1、dfs过程中，dfs到v点，需要在v节点的深度范围内，表示完v节点 2^j 的祖先
- 2、给定u,v,如何快速找到最近公共祖先的一个转换。
- 3、找到u,v的祖先k后，如何快速求出u—k和v—k的最值。
- 4、核心用到了深度T与 2^j 的“&”运算，即

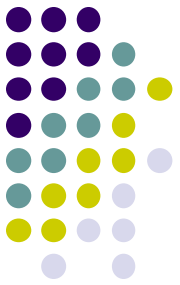
$$T \& (2^{<< j})$$

倍增必写练习题目

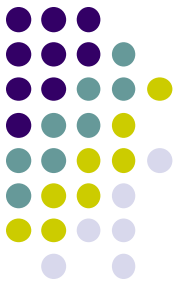


- 落谷： **P3379** 模板题
- 落谷： **P2680** 暂时不写。
- 落谷： **P1967** (noip 2013 day1-3)

练习题目



- [hdu 2586 How far away ?](#)
- LCA模板题
- 题意：给一个无根树，有q个询问，每个询问两个点，问两点的距离。求出 $lca = LCA(X, Y)$ ，然后 $dir[x] + dir[y] - 2 * dir[lca]$
- $dir[u]$ 表示点u到树根的距离



- [poj 1986 Distance Queries](#)
- LCA
- 题意：LCA模板题，输入 n 和 m ，表示 n 个点 m 条边，下面 m 行是边的信息，两端点和权，后面的那个字母无视掉，没用的。接着 k ，下面 k 个询问lca，输出即可
- 有人说要考虑不连通的情况，我没考虑AC了，另外可能有 u ， u 这样的询问，不过这不影响，照样是写模板，没有特判，一样能过

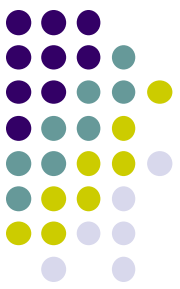
poj 3694 Network

连通分量 + LCA : 先缩点, 再求LCA, 并且不断更新, 这题用了朴素方法来找LCA, 并且在路径上做了一些操作

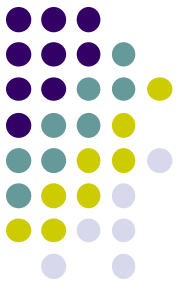


- poj 3417 Network

- LCA + Tree DP : 在运行Tarjan处理完所有的LCA询问后, 进行一次树DP, 树DP不难, 但是要想到用树DP并和这题结合还是有点难度



- [poj 3728 The merchant](#)
- LCA + 并查集的变形，优化：好题，难题，思维和代码实现都很有难度，需要很好地理解Tarjan算法中并查集的本质然后灵活变形，需要记录很多信息（有点dp的感觉）
- [hdu 3830 Checkers](#)
- LCA + 二分：好题，有一定思维难度。先建立起二叉树模型，然后将要查询的两个点调整到深度一致，然后二分LCA所在的深度，然后检验



练习题目参考博客

<http://www.cnblogs.com/scau20110726/archive/2013/06/14/3135095.html>

Tarjan LCA 学习参考博客

<http://alanyume.com/130.html>

<https://www.cnblogs.com/hulean/p/11144059.html>

路径问题分析



1、简单路径求极值，求和。

路径极值：只能用树上倍增算法

路径求和：可以用**RMQ**或者**tarjan**算法

2、边更新、边求极值，求和。

树链剖分中的点剖、边剖

（应用线段树的知识）

最近公共祖先